Knowledge Graph Generation Toolkit

Data Engineering Team

June 18, 2026

Overview

This document contains a comprehensive toolkit of scripts utilized for generating a medical Knowledge Graph from the Kurdish medical dataset. It leverages Ollama for semantic relation extraction and Neo4j for storage. Below is a summary table containing links to the code for each step in the pipeline, alongside an explanation of its purpose.

Script Name	**Section Link**	**Purpose / Why We Use It**
Knowledge Graph Pipeline		
KG Generator	Section 1	Extracts medical triples using Ollama and stores them in Neo4j.
Run KG Script	Section 2	Shell script to run the knowledge graph generator in the background.
Setup & Environment		
Setup Script	Section 3	Prepares the environment by installing dependencies and configuring Neo4j/Ollama.
Setup Runner	Section 4	Python wrapper to execute the setup shell script in the background.
Model Loader	Section 5	A dedicated script to manage loading the Ollama model into GPU memory.
Requirements	Section 6	Lists the necessary Python packages for the pipeline.

Table 1: Knowledge Graph generation and setup scripts categorized by their functional domain.

1 Knowledge Graph Generator

Purpose: We use this script to extract medical triples from the Kurdish dataset using the Ollama service and store them dynamically into a Neo4j database. It includes retry mechanisms and checkpointing.

Listing 1: kg_generator.py

```python
import json
import os
import re
import logging
from typing import List, Dict
from tqdm import tqdm
from ollama import Client
from neo4j import GraphDatabase
from dotenv import load_dotenv

# Load environment variables
load_dotenv()

# Setup Logging
logging.basicConfig(
    level=logging.INFO,
    format='%(asctime)s - %(levelname)s - %(message)s',
    handlers=[
        logging.FileHandler("kg_pipeline.log"),
        logging.StreamHandler()
    ]
)

# Configuration
OLLAMA_MODEL = "gemma4:31b"
NEO4J_URI = os.getenv("NEO4J_URI", "bolt://localhost:7687")
NEO4J_USER = os.getenv("NEO4J_USER", "neo4j")
NEO4J_PASSWORD = os.getenv("NEO4J_PASSWORD", "password")
DATASET_PATH = "kurdish_medical_corpus_kmc.json"
CHECKPOINT_PATH = "processed_ids.txt"
OUTPUT_FILE = "extracted_triples.jsonl"

class KMCKnowledgeGraph:
    def __init__(self):
        self.client = Client(host=os.getenv("OLLAMA_HOST", "http://localhost:11434"))
        try:
            self.driver = GraphDatabase.driver(NEO4J_URI, auth=(NEO4J_USER,
    NEO4J_PASSWORD))
            # Verify connection
            self.driver.verify_connectivity()
            logging.info("Connected to Neo4j successfully.")
        except Exception as e:
            logging.error(f"Failed to connect to Neo4j: {e}")
            raise

        self.processed_ids = self._load_checkpoints()

    def _load_checkpoints(self) -> set:
        if os.path.exists(CHECKPOINT_PATH):
            with open(CHECKPOINT_PATH, 'r') as f:
```

```
                return set(line.strip() for line in f)
        return set()

    def _save_checkpoint(self, entry_id: str):
        with open(CHECKPOINT_PATH, 'a') as f:
            f.write(f"{entry_id}\n")
        self.processed_ids.add(entry_id)

    def close(self):
        self.driver.close()

    def clean_text(self, text: str) -> str:
        if not text:
            return ""
        text = re.sub(r'\s+', ' ', text)
        text = text.strip()
        return text

    def extract_triples(self, text: str, max_retries: int = 5) -> List[Dict]:
        """
        Extracts triples with entity types for better Neo4j labeling.
        Implements a retry mechanism to ensure high-quality, non-empty medical data.
        """
        # Improved prompt with semantic guidance and quality focus
        prompt = f"""
        You are an elite Kurdish medical linguist and knowledge graph engineer.
        Your task is to extract high-quality, semantically rich medical triples from the provided Kurdish text.

        CRITICAL QUALITY REQUIREMENTS:
        1. LANGUAGE: All medical entities and relationships MUST be in natural, accurate Kurdish (Sorani/Central Kurdish).
        2. MEANINGFUL RELATIONS: Avoid generic relations like 'has' or 'is'. Use precise medical verbs.
           Examples:
           - 'هۆی دەبێتە' (causes)
           - 'بۆ نیشانەیە' (is a symptom of)
           - 'لە دەستنیشانکراوە یەکەم جار بۆ' (first diagnosed in)
           - 'چارەسەری بۆ بەکاردێت' (used for treatment of)
           - 'سەر کاردەکاتە' (affects)
           - 'بەشێکە لە' (is part of)
        3. ENTITY TYPES: 'head_type' and 'tail_type' MUST be exactly one of these English labels: [Disease, Symptom, Treatment, Medication, Anatomy, Provider, Organization].
        4. DATA INTEGRITY: Every triple MUST have a valid, descriptive 'head', 'relation', and 'tail'. No empty strings or generic placeholders.
        5. QUALITY ASSESSMENT: Only extract facts that are clearly stated. If a triple feels ambiguous or poor quality for the Kurdish language, discard it.

        Kurdish Text: {text}

        Requested JSON Format (List of Objects):
        [
            {{
                "head": "Entity Name in Kurdish",
                "head_type": "English Label",
                "relation": "Detailed Kurdish Relationship",
                "tail": "Target Entity in Kurdish",
                "tail_type": "English Label"
```

```
            }}
        ]

        Return ONLY valid JSON. If the text contains no meaningful medical relationships, return an empty list [].
        """

        for attempt in range(max_retries):
            try:
                response = self.client.generate(
                    model=OLLAMA_MODEL,
                    prompt=prompt,
                    format="json",
                    options={
                        "temperature": 0.3, # Increased for more descriptive relations
                        "top_p": 0.9,
                        "num_predict": 2048
                    }
                )

                if isinstance(response, dict):
                    content = response.get('response', '')
                else:
                    content = getattr(response, 'response', '')

                if not content:
                    continue

                if "```" in content:
                    match = re.search(r'(\[.*\]|\{.*\})', content, re.DOTALL)
                    if match:
                        content = match.group(1)

                try:
                    data = json.loads(content)
                except json.JSONDecodeError:
                    logging.warning(f"Attempt {attempt+1}: JSON Parse Error. Retrying ...")
                    continue

                # Normalize to list
                if isinstance(data, dict):
                    raw_triples = [data]
                elif isinstance(data, list):
                    raw_triples = [item for item in data if isinstance(item, dict)]
                else:
                    continue

                # STRICT VALIDATION PASS
                valid_triples = []
                is_perfect_batch = True

                for t in raw_triples:
                    head = str(t.get('head', '')).strip()
                    tail = str(t.get('tail', '')).strip()
                    relation = str(t.get('relation', '')).strip()

                    # Check for empty values or generic placeholders
                    if not head or not tail or not relation or len(head) < 2 or len(
```

```
    tail) < 2:
159                     logging.info(f"Attempt {attempt+1}: Detected poor quality
    triple. Retrying batch...")
160                     is_perfect_batch = False
161                     break
162
163                 valid_triples.append({
164                     "head": head,
165                     "head_type": t.get('head_type', 'Entity'),
166                     "relation": relation,
167                     "tail": tail,
168                     "tail_type": t.get('tail_type', 'Entity')
169                 })
170
171             if is_perfect_batch and valid_triples:
172                 # Final check: are the relations meaningful?
173                 # If the LLM just gave us "has" or "is", we might want to nudge it
    ,
174                 # but for now we accept non-empty descriptive text.
175                 return valid_triples
176
177             if not raw_triples and attempt == 0:
178                 # If it's the first try and it found nothing, it might really be
    empty
179                 return []
180
181         except Exception as e:
182             logging.warning(f"Attempt {attempt+1} Exception: {e}")
183
184     return []
185
186 def store_triple(self, triple: Dict):
187     """
188     Stores triples with dynamic labels.
189     """
190     try:
191         head = triple.get('head')
192         h_type = triple.get('head_type', 'Entity')
193         rel = triple.get('relation', 'RELATED_TO').upper().replace(" ", "_")
194         tail = triple.get('tail')
195         t_type = triple.get('tail_type', 'Entity')
196
197         if not head or not tail:
198             return
199
200         # Sanitize labels (Neo4j labels must be alphanumeric and not start with
    digits)
201         h_label = re.sub(r'[^a-zA-Z0-9]', '', str(h_type)) or "Entity"
202         if h_label[0].isdigit(): h_label = "L_" + h_label
203
204         t_label = re.sub(r'[^a-zA-Z0-9]', '', str(t_type)) or "Entity"
205         if t_label[0].isdigit(): t_label = "L_" + t_label
206
207         # Sanitize relation
208         rel = re.sub(r'[^a-zA-Z0-9_]', '', str(rel)) or "RELATED_TO"
209         if rel[0].isdigit(): rel = "R_" + rel
210
211         with self.driver.session() as session:
212             # Use dynamic labels in Cypher
```

```
                query = (
                    f"MERGE (h:{h_label} {{name: $head}}) "
                    f"MERGE (t:{t_label} {{name: $tail}}) "
                    f"MERGE (h)-[r:{rel}]->(t) "
                    "RETURN h, r, t"
                )
                session.run(query, head=head, tail=tail)
        except Exception as e:
            logging.error(f"Failed to store triple in Neo4j: {triple}. Error: {e}")

    def process_dataset(self):
        logging.info(f"Loading dataset from {DATASET_PATH}...")
        try:
            with open(DATASET_PATH, 'r', encoding='utf-8') as f:
                data = json.load(f)
        except Exception as e:
            logging.error(f"Could not load JSON: {e}")
            return

        logging.info(f"Processing {len(data)} entries. Found {len(self.processed_ids)} already processed.")

        for entry in tqdm(data):
            entry_id = entry.get('id')
            if not entry_id or entry_id in self.processed_ids:
                continue

            raw_text = entry.get('response', '')
            if not raw_text:
                self._save_checkpoint(entry_id)
                continue

            cleaned_text = self.clean_text(raw_text)
            triples = self.extract_triples(cleaned_text)

            if triples:
                # Save to locally to JSONL file for inspection
                try:
                    with open(OUTPUT_FILE, 'a', encoding='utf-8') as f_out:
                        record = {"id": entry_id, "triples": triples}
                        f_out.write(json.dumps(record, ensure_ascii=False) + "\n")
                except Exception as e:
                    logging.error(f"Failed to write to {OUTPUT_FILE}: {e}")

                for triple in triples:
                    self.store_triple(triple)

            self._save_checkpoint(entry_id)

        logging.info("Pipeline finished successfully.")

if __name__ == "__main__":
    kg = KMCKnowledgeGraph()
    try:
        kg.process_dataset()
    except KeyboardInterrupt:
        logging.info("Process interrupted by user. Progress saved.")
    except Exception as e:
        logging.error(f"Fatal error: {e}")
```

```
    finally:
        kg.close()
```

2 Run KG Script

Purpose: A convenience shell script designed to activate the virtual environment and execute the knowledge graph generator in the background via `nohup`.

Listing 2: run_kg.sh

```
#!/bin/bash

# Navigate to the script directory (optional, usually good practice)
# cd "$(dirname "$0")"

# Check if virtual environment exists and activate it
if [ -d ".venv" ]; then
    echo "Activating virtual environment..."
    source .venv/bin/activate
elif [ -d "venv" ]; then
    echo "Activating virtual environment (venv)..."
    source venv/bin/activate
fi

# Run the kg_generator.py in the background
# We use nohup to keep it running after logout
# We redirect output to kg_console.log
nohup python3 kg_generator.py > kg_console.log 2>&1 &

PID=$!
echo "--------------------------------------------------"
echo "Knowledge Graph Generator started in background."
echo "PID: $PID"
echo "Console output is being redirected to: kg_console.log"
echo "Application logs are in: kg_pipeline.log"
echo "To stop the process, run: kill $PID"
echo "--------------------------------------------------"
```

3 Setup Script

Purpose: We use this shell script to prepare the environment, install required Python dependencies from `requirements.txt`, initialize a Neo4j Docker container, and verify the Ollama installation and model availability.

Listing 3: setup.sh

```
#!/bin/bash

# KMC Knowledge Graph Setup Script

```

```
echo "🚀 Starting setup for KMC Knowledge Graph Pipeline..."

# 1. Check for Python
if ! command -v python3 &> /dev/null; then
    echo "❌ Python3 is not installed. Please install it first."
    exit 1
fi

# 2. Install Requirements
echo "📦 Installing dependencies from requirements.txt..."
pip install --upgrade pip
pip install -r requirements.txt

# 3. Docker and Neo4j Setup
echo "🐳 Checking Docker status..."
if ! command -v docker &> /dev/null; then
    echo "❌ Docker not found. Installing Docker..."
    curl -fsSL https://get.docker.com -o get-docker.sh
    sudo sh get-docker.sh
    sudo usermod -aG docker $USER
    rm get-docker.sh
    DOCKER_CMD="sudo docker"
    echo "✅ Docker installed. Using 'sudo docker' for this session."
else
    DOCKER_CMD="docker"
fi

echo "🐳 Setting up Neo4j container..."
if ! $DOCKER_CMD ps -a --format '{{.Names}}' | grep -q "^neo4j-kmc$"; then
    $DOCKER_CMD run -d \
        --name neo4j-kmc \
        -p 7474:7474 -p 7687:7687 \
        -e NEO4J_AUTH=neo4j/password \
        -e NEO4J_PLUGINS='["apoc"]' \
        --restart always \
        neo4j:latest
    echo "✅ Neo4j started. Access it at http://localhost:7474"
else
    echo "✅ Neo4j container 'neo4j-kmc' already exists."
    if [ "$($DOCKER_CMD inspect -f '{{.State.Running}}' neo4j-kmc)" != "true" ]; then
        echo "🔄 Starting Neo4j container..."
        $DOCKER_CMD start neo4j-kmc
    fi
fi

# 4. Check for Ollama
if ! command -v ollama &> /dev/null; then
    echo "❌ Ollama not found in PATH. Please install it from https://ollama.com"
else
    if ollama list | grep -q "gemma4:26b"; then
        echo "✅ Model gemma4:26b already exists in Ollama."
    else
        echo "🤖 Pulling Gemma model (gemma4:26b)..."
        ollama pull gemma4:26b
    fi
fi

# 5. Create .env template if it doesn't exist
if [ ! -f .env ]; then
```

```
    echo "📝 Creating .env template..."
    echo "NEO4J_URI=bolt://localhost:7687" > .env
    echo "NEO4J_USER=neo4j" >> .env
    echo "NEO4J_PASSWORD=password" >> .env
fi

echo "✅ Setup complete!"
echo "🚀 You can now run the pipeline: 'python kg_generator.py'"
```

4 Setup Runner

Purpose: A Python utility script to launch the `setup.sh` installation routine in a detached background process, redirecting its output to a log file.

Listing 4: run_setup.py

```python
import subprocess
import os
import sys

def run_setup_background():
    """
    Runs setup.sh in the background and redirects all output to setup.log.
    Designed for execution on an Ubuntu server.
    """
    script_name = "setup.sh"
    log_name = "setup.log"

    # Check if setup.sh exists
    if not os.path.exists(script_name):
        print(f"❌ Error: {script_name} not found in the current directory.")
        return

    # Ensure the script is executable
    os.chmod(script_name, 0o755)

    print(f"🚀 Launching {script_name} in the background...")
    print(f"📄 Output will be logged to {log_name}")

    try:
        # Open the log file for writing
        with open(log_name, "w") as log_file:
            # subprocess.Popen starts the process without waiting for it to finish
            # We use 'bash' explicitly to ensure it runs correctly on Ubuntu
            process = subprocess.Popen(
                ["bash", script_name],
                stdout=log_file,
                stderr=subprocess.STDOUT, # Redirect stderr to the same log file
                start_new_session=True    # Detach from the current terminal session
            )

        print(f"✅ Setup background process started successfully!")
        print(f"🆔 PID: {process.pid}")
        print(f"💡 You can monitor progress with: tail -f {log_name}")

```

```
    except Exception as e:
        print(f"❌ Failed to start the setup process: {e}")

if __name__ == "__main__":
    run_setup_background()
```

5 Model Loader

Purpose: A sophisticated bash script to trigger the loading of an Ollama model into specific GPU memory in the background and verify its active status via polling.

Listing 5: load_model_gpu_1.sh

```
#!/bin/bash

# ================================================================================
# Ollama Model GPU Loader & Verification Script
# ================================================================================
# Description: This script triggers the loading of an Ollama model into GPU
#              memory in the background and verifies the status.
# Author: Antigravity AI
# ================================================================================

# --- Configuration ---
# You can change these variables to match your environment
MODEL_NAME="gemma4:26b"
LOG_FILE="ollama_gpu_1_load.txt"
GPU_ID=1  # The index of the GPU to use
OLLAMA_PORT=11435

# ANSI Color Codes for Premium UI
GREEN='\033[0;32m'
BLUE='\033[0;34m'
CYAN='\033[0;36m'
YELLOW='\033[1;33m'
RED='\033[0;31m'
BOLD='\033[1m'
NC='\033[0m' # No Color

# Header
echo -e "${CYAN}${BOLD}============================================================${NC}"
echo -e "${CYAN}${BOLD}       OLLAMA GPU LOAD & TEST UTILITY                       ${NC}"
echo -e "${CYAN}${BOLD}============================================================${NC}"

# Initialize log file
echo "--- Ollama GPU Load Session: $(date) ---" > "$LOG_FILE"

# --- Function: Ensure Server is Running ---
ensure_server_running() {
    echo -e "${YELLOW}[*] Action: Checking Ollama server status...${NC}"

    # Check if reachable
    if curl -s "http://localhost:$OLLAMA_PORT/api/tags" > /dev/null 2>&1; then
        echo -e "${GREEN}[+] Ollama server is already active.${NC}"
        return 0
```

```
    fi

    echo -e "${YELLOW}[!] Ollama server not detected. Attempting to start...${NC}"

    # Check if the user's custom start script exists in the same directory
    if [ -f "./start_ollama_server_second.sh" ]; then
        echo -e "${CYAN}[i] Found 'start_ollama_server_second.sh'. Using it to initialize...${NC}"
        # Run in background
        nohup bash ./start_ollama_server_second.sh >> "$LOG_FILE" 2>&1 &
    else
        echo -e "${CYAN}[i] No start script found. Running 'ollama serve' directly...${NC}"
        # Set basic environment just in case
        export OLLAMA_HOST="0.0.0.0:$OLLAMA_PORT"
        nohup ollama serve >> "$LOG_FILE" 2>&1 &
    fi

    # Wait and verify status
    echo -n "Waiting for API availability"
    for i in {1..20}; do
        echo -n "."
        if curl -s "http://localhost:$OLLAMA_PORT/api/tags" > /dev/null 2>&1; then
            echo -e "\n${GREEN}[SUCCESS] Ollama server is up and responding!${NC}" | tee -a "$LOG_FILE"
            return 0
        fi
        sleep 2
    done

    echo -e "\n${RED}[ERROR] Server failed to start within 40 seconds.${NC}" | tee -a "$LOG_FILE"
    exit 1
}

# --- Function: Load Model ---
load_to_gpu_bg() {
    echo -e "${YELLOW}[*] Action: Loading '$MODEL_NAME' to GPU $GPU_ID in background...${NC}"

    # Trigger loading using 'ollama run' with an empty input
    # We must set OLLAMA_HOST to talk to the correct port (11435)
    export OLLAMA_HOST="localhost:$OLLAMA_PORT"
    nohup bash -c "export CUDA_VISIBLE_DEVICES=$GPU_ID; export OLLAMA_HOST=localhost:$OLLAMA_PORT; ollama run $MODEL_NAME ''" >> "$LOG_FILE" 2>&1 &

    BG_PID=$!
    echo -e "${GREEN}[+] Load process dispatched (PID: $BG_PID).${NC}" | tee -a "$LOG_FILE"
    echo -e "${CYAN}[i] Logging output to: $LOG_FILE${NC}"
}

# --- Function: Verification Test ---
run_verification_test() {
    echo -e "\n${YELLOW}[*] Action: Verifying GPU VRAM status...${NC}"
    echo -n "Waiting for model initialization"

    # Polling for up to 60 seconds (12 attempts * 5 seconds)
    MAX_ATTEMPTS=12
```

```
    SUCCESS=false

    for ((i=1; i<=MAX_ATTEMPTS; i++)); do
        echo -n "."
        # Check 'ollama ps' on the specific host/port
        if OLLAMA_HOST="localhost:$OLLAMA_PORT" ollama ps 2>/dev/null | grep -q "$MODEL_NAME"; then
            echo -e "\n${GREEN}${BOLD}[SUCCESS] Model '$MODEL_NAME' is active in GPU memory!${NC}" | tee -a "$LOG_FILE"
            SUCCESS=true
            break
        fi
        sleep 5
    done

    if [ "$SUCCESS" = true ]; then
        echo -e "${BLUE}------------------------------------------------------------${NC}"
        echo -e "${BOLD}Current GPU Load Status (Port $OLLAMA_PORT):${NC}"
        OLLAMA_HOST="localhost:$OLLAMA_PORT" ollama ps | grep -E "NAME|${MODEL_NAME}"
        echo -e "${BLUE}------------------------------------------------------------${NC}"
        return 0
    else
        echo -e "\n${RED}${BOLD}[FAILURE] Model verification timed out.${NC}" | tee -a "$LOG_FILE"
        echo -e "${YELLOW}[!] Check '$LOG_FILE' for potential error messages.${NC}"
        return 1
    fi
}

# --- Main Execution ---
ensure_server_running
load_to_gpu_bg
run_verification_test

echo -e "\n${CYAN}Done.${NC}"
```

6 Requirements

Purpose: Defines the required Python packages for running the knowledge graph pipeline.

Listing 6: requirements.txt

```
neo4j
ollama
python-dotenv
tqdm
```